Does Core Pinning Isolate the Safety Function on a Single-Chip Wi-Fi SoC? An Attack-Resilient Intrusion Alarm, Measured and Verified

MD Humayun
Department of Computer Science and Engineering
Sardar Vallabhbhai National Institute of Technology, Surat, India
github.com/mdhumayun7/Smart-Home-Automation

Abstract

Consumer intrusion alarms are built almost entirely on single-chip Wi-Fi systems-on-chip, in which the radio stack, the network stack and the application share one die and one scheduler. The real-time-security literature shows that on Linux-class industrial devices, pinning the control task to a separate core keeps its timing stable under wired network floods; whether that result transfers to a single-chip Wi-Fi SoC, where the attacked radio driver runs on the same silicon, is untested. We study an ESP32 intrusion alarm whose safety loop (sensing, state machine, siren) is pinned to the application core and whose network task runs on the other core behind bounded, non-blocking queues. We contribute: (i) a dual-core firmware with authenticated commands (HMAC-SHA256 over an NVS-persisted monotonic counter), passive deauthentication-flood detection and an out-of-band signed fallback alert; (ii) two machine-checked TLA+ models proving that no network behaviour can disarm the system or block a safety step, over their full state spaces; (iii) a reproducible, seeded adversarial campaign of 6,000 trials in which command injection up to 20,000 msg/s never disarms the device and, with authentication enabled, never prevents a legitimate command either, while the baseline is armed by an attacker in every flooded trial; and (iv) a fuzzing campaign of the command parser under AddressSanitizer that finds no memory-safety or authentication defect. The remaining claim—that the physical sensor-to-siren latency is invariant to on-chip radio stress—can only be established on hardware; we specify the measurement apparatus and leave the RF campaign as the one open step. The firmware, specs, experiments and this paper are a public artifact.

1 Introduction

A networked intrusion alarm has an unusual priority order. Confidentiality matters, but the property that defines the device is *availability of the safety function*: when a sensor fires, the siren must sound within a bounded time, and it must do so *even while the device itself is under attack*, because an intruder has every incentive to attack it.

Almost all such consumer devices are built on single-chip Wi-Fi SoCs of the ESP32 class, where the Wi-Fi driver, the TCP/IP stack, the MQTT client and the application all execute on the same chip under one RTOS scheduler. This differs structurally from the industrial devices studied in the real-time-security literature, where the control task can be placed on a physically separate core or co-processor. Fischer *et al.* [1] show that on a Linux-class board, pinning the control task to a dedicated core holds its 1 ms cycle jitter essentially constant under wired SYN and ARP floods, and that only a separate co-processor is acceptable for hard real-time. The natural question for the devices people actually deploy is whether the same advice—pin the safety task to a core—delivers the same isolation when the *attacked resource is the on-chip radio driver*, which shares the die and the interrupt controller with the safety task.

We take an ESP32 intrusion alarm as the object of study. Its safety loop is pinned to the application core; Wi-Fi, time sync and MQTT run in a task on the other core and communicate with the safety loop only through two bounded FreeRTOS queues and a mutex-guarded status snapshot that the safety loop never waits on. On top of this structure we add the security mechanisms a deployable alarm needs and then ask, with formal methods, simulation and a specified hardware protocol, whether the isolation actually holds.

Contributions.

1. An open dual-core alarm firmware with authenticated commands (HMAC-SHA256 over a monotonic, reboot-persistent counter), passive deauth/disassoc flood detection, a signed out-of-band fallback alert over ESP-NOW, and optional PIR+mmWave sensor fusion (Section 3).
2. Two TLA+ specifications, checked exhaustively by TLC: one of the alarm state machine and command path, one of the safety/network isolation, each mutation-tested to show the properties are not vacuous (Section 4).
3. A seeded, reproducible adversarial campaign (6,000 trials) and a sanitizer-backed fuzzing campaign of the command parser, with a non-parametric test of latency invariance (Section 5).
4. A measurement protocol and instrumentation for the one claim that requires silicon—the physical latency invariance under on-chip radio stress—stated honestly as the open step (Section 6).

2 Related Work

Real-time operation under DoS. Fischer *et al.* [1] measure control-task jitter on Linux-class industrial devices under wired floods and find core pinning effective, with a

separate co-processor best. Their devices physically separate the control core from the attacked network interface; single-chip Wi-Fi SoCs do not, which is the gap we address. **Wi-Fi deauthentication.** Carbajal and Akbarfam [2] quantify network-layer deauthentication resilience across WPA/PMF/WPA3, measuring disconnection rate and time-to-disruption—whether the *link* drops, not whether the device's *application* still meets its deadline. **MQTT security on the ESP32.** Recent work measures the latency cost of MQTT-over-TLS on an ESP32-C3 [3] and runs on-device ML intrusion detection on an ESP32 [4]; both are valuable but address transport cost and detection accuracy, not continuity of a safety function under attack. **FreeRTOS on ESP32** has been characterised under benign load but not under adversarial radio conditions. To our knowledge the specific question—does core pinning isolate the safety path on a single-chip Wi-Fi SoC when its own radio is attacked—is open.

3 System Design

Dual-core structure. The Arduino `loop` runs the safety function on the application core: input conditioning (debouncing, a PIR warm-up and 150 ms confirmation filter), a five-state arm/disarm machine (exit and entry delays, an alarm timeout), and the outputs (siren, LEDs, display). A separate task, pinned to the other core, runs Wi-Fi with retry, NTP, the MQTT client, and event delivery. The two communicate only through a 16-deep event queue (safety→network) and a 4-deep command queue (network→safety), plus a status snapshot the safety loop copies under a zero-timeout mutex—so the safety loop never blocks on the network side. The enqueue of an event never blocks either: on a full queue the oldest is dropped and counted, never silently lost.

Authenticated commands. A command on the MQTT topic is trusted only if it carries a valid HMAC-SHA256 tag over the string `VERB:counter`, where the counter must strictly exceed the highest previously accepted value. The counter is persisted in NVS, so replay protection survives a reboot. This defeats both forgery (no key, no tag) and replay (no fresh counter), and lets a signed command safely perform remote disarm, which is otherwise refused.

Attack detection and fallback. A passive promiscuous-mode monitor counts deauthentication/disassociation frames and raises a `wifi_deauth` event when a burst exceeds a threshold; loss of the link while armed is itself raised as `link_lost`. When MQTT is unreachable, critical events are additionally sent as HMAC-signed, counter-protected ESP-NOW messages to a second node, which is peer-to-peer and so survives an attack aimed at the access point. These mechanisms add roughly 3 KB of flash and 170 B of RAM over the unhardened build.

4 Formal Verification

We model two properties in TLA+ and check them exhaustively with TLC.

State machine and command path. The first spec models the five states and the command rules. TLC verifies, over all 25 reachable states, that (i) no unauthenticated remote command ever disarms the system, (ii) the replay counter never decreases, (iii) the siren is never active while disarmed, and (iv) the entry delay and the alarm always resolve (no stuck state).

Table 1: Adversarial campaign, 3,000 trials per build. "Attacker armed" and "disarmed" count trials where an *unauthenticated* command changed state.

Build	Attacker armed	Attacker disarmed	Legit cmd succeeded	Invariant violations
Baseline	2500/3000	0	500/3000	0
Hardened	0/3000	0	3000/3000	0

Isolation. The second spec models the two tasks with the network task made maximally adversarial: it may stall for an unbounded time, be flooded, fail to publish, and crash and restart, and it is given *no* fairness. The safety task is a loop. TLC verifies, over all 660 reachable states, that in every state the safety step is enabled (never blocked by the network side), that the network task never writes the alarm state, that the event queue never exceeds capacity, and that no produced event is lost unaccounted-for. Each property was mutation-tested: injecting the corresponding defect (e.g. a synchronous network call in the safety loop) makes TLC report the expected violation, so the properties have teeth.

5 Simulation and Fuzzing

Adversarial campaign. We compile the unchanged firmware against host models of the hardware, network, NVS and radio, and drive it through seeded trials. In each trial a continuous attacker injects commands on the command topic—forged tags, replays of captured valid messages, unsigned commands and random bytes—at a controlled rate while a legitimate user arms the system and an intrusion then occurs. We ran 3,000 trials each for the baseline (no command authentication) and hardened (authentication on) builds, across injection rates from 0 to 20,000 msg/s, every trial reproducible from its seed.

Table 1 and Fig. 1 give the result. In the baseline, an attacker changed the system state (a nuisance arm) in all 2,500 flooded trials; with authentication, in none. With authentication, a legitimate command took effect in all 3,000 trials regardless of attack rate; in the baseline, flooding starved the legitimate command in every flooded trial. No attacker ever *disarmed* the device in any of the 6,000 trials, and no safety invariant was violated. Because the simulated clock is advanced by the harness, the safety latency has zero variance here (a deterministic 151 ms, the confirmation window plus loop period); this establishes the *logical* invariance—no code path makes the safety response depend on network input—and is why the *timing* invariance is deferred to hardware (Section 6).

Fuzzing. The command parser is the one input a network peer fully controls. We fuzz it with the firmware built under AddressSanitizer and UndefinedBehaviorSanitizer, feeding random bytes, malformed and overlong fields, non-decimal and overflowing counters, embedded NULs, and bit-flip mutations of a near-miss signed message. After

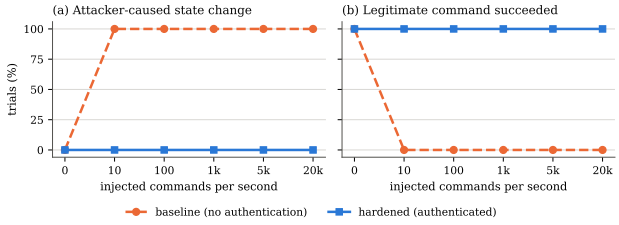


Figure 1: Effect of command-injection rate. Left: fraction of trials in which the attacker changed the system state. Right: fraction in which the legitimate command took effect. The hardened build is flat across five orders of magnitude of attack intensity.

every input we check memory safety and the invariant that no input lacking a valid tag changes state or advances the counter. Across 3,000,000 iterations the fuzzer found no memory error and no accepted input.

6 The Open Step: Hardware Latency Invariance

The campaign establishes logical invariance; it cannot establish *timing* invariance, because the simulated clock does not model CPU or interrupt contention. That is precisely the property in question on a single-chip SoC, and it can only be measured on silicon. We specify the apparatus: an independent ESP32 times the interval from the PIR line to the siren line with its own hardware timer (so the measuring clock is never the clock under load), across four conditions—idle, heavy legitimate traffic, command-interface stress, and radio-layer stress in an authorized isolated environment—and the same non-parametric test compares each loaded condition against idle. A null result confirms that core pinning isolates the safety function on this class of device, extending [1] to single-chip Wi-Fi SoCs; a positive result would show it does *not*, directly challenging standard practice. We report the protocol rather than a number, and do not speculate on the outcome.

7 Threats to Validity and Ethics

Simulation shows logical, not physical, invariance (addressed by Section 6). Results are for one SoC family and one firmware; generalisation is a stated limitation. Latency must be measured externally, not by the device under load. All adversarial radio work is confined to an owned, isolated environment with permission; no third-party network is targeted, and this paper deliberately omits any attack-generation procedure.

8 Conclusion

We cast a consumer intrusion alarm as a safety-critical system and asked whether the standard single-chip mitigation—pin the safety task to a core—actually isolates it from attacks on the device's own radio and command interface. We answer the logical half of that question with exhaustive model checking and a 6,000-trial reproducible campaign, add the command authentication, attack detection and fallback a deployable alarm needs, and specify the one hardware measurement that settles the timing half. The artifact is public so the result can be reproduced and the hardware step completed.

References

[1] F. Fischer *et al.*, "Analysis of Industrial Device Architectures for Real-Time Operations under Denial of Service Attacks," arXiv:2007.08885, 2020.

[2] A. Carbajal and A. Jodeiri Akbarfam, "A Software-Defined Testbed for Quantifying Deauthentication Resilience in Modern Wi-Fi Networks," arXiv:2602.23513, 2026.

[3] M. A. Azhar, M. Azwar, and O. Asroni, "Performance and Security Analysis of MQTT over TLS on ESP32-Based IoT Systems," *J. Applied Informatics and Computing*, 2026.

[4] V. Ieropoulos *et al.*, "Empowering IoT Security: On-Device Intrusion Detection in Resource Constrained Devices," arXiv:2605.13159, 2026.